

Algorithms 2

Approximating Vertex Cover

Daniel Rosenberg & Michael Trushkin

Ariel University

1. Vertex Cover

Definition 1 Vertex Cover (VC)

Given: An undirected graph $G = (V, E)$.

Find: A minimum set $C \subseteq V$ such that every edge $e \in E$ has at least one endpoint in C .

Definition 2 Vertex Cover (VC)

Given: An undirected graph $G = (V, E)$.

Find: A minimum set $C \subseteq V$ such that every edge $e \in E$ has at least one endpoint in C .

- VC is NP-hard (one of Karp's 21 NP-complete problems)
- We seek a polynomial-time **approximation algorithm**

Definition 3 Vertex Cover (VC)

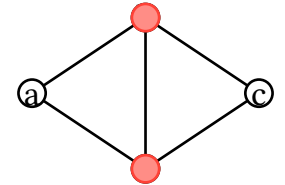
Given: An undirected graph $G = (V, E)$.

Find: A minimum set $C \subseteq V$ such that every edge $e \in E$ has at least one endpoint in C .

- VC is NP-hard (one of Karp's 21 NP-complete problems)
- We seek a polynomial-time **approximation algorithm**

Observation 3 The complement $V \setminus C$ of a vertex cover is an **independent set**, and vice versa. So

C minimum VC $\Leftrightarrow V \setminus C$ maximum independent set.



The natural greedy algorithm fails to give a constant approximation ratio:

1.2 Failed Greedy Approaches

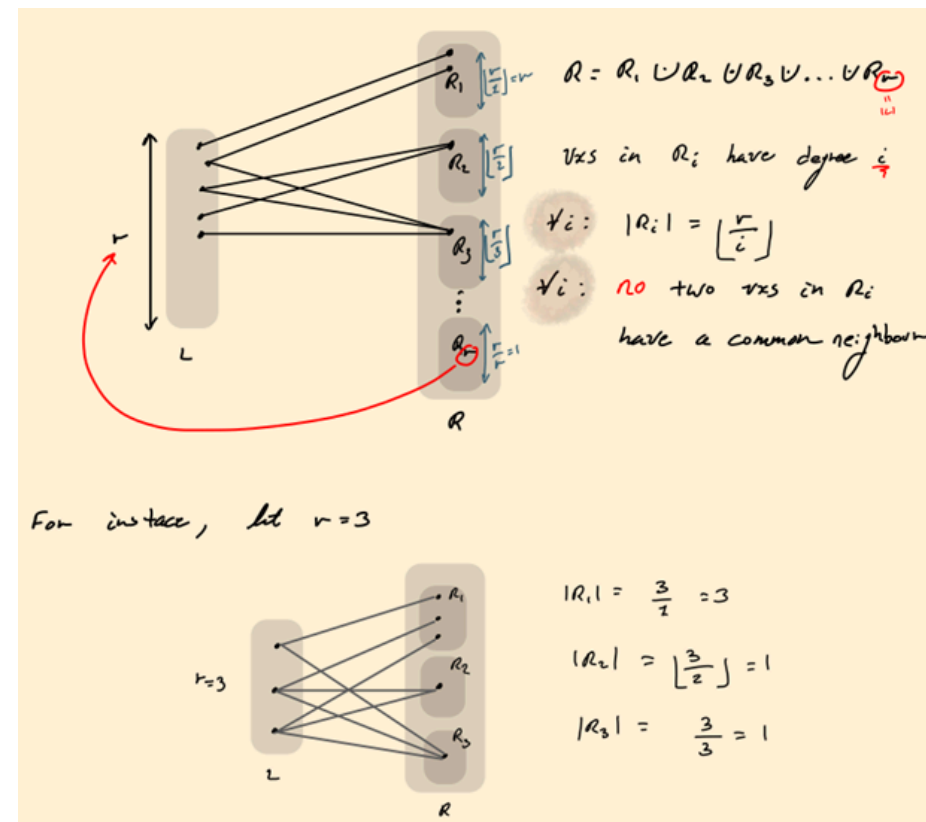
The natural greedy algorithm fails to give a constant approximation ratio:

Greedy 1: Pick highest-degree vertex repeatedly

Algorithm 2: Greedy-Degree

```
1: while  $E \neq \emptyset$  do
2:    $v \leftarrow \arg \max_{u \in V} \deg(u)$ 
3:   Add  $v$  to  $C$ ; remove  $v$  and all incident edges
4: end
5: return  $C$ 
```

1. Vertex Cover
Consider the following construction of graph R with r .



Consider the following construction of graph R with r .

1.2 Failed Greedy Approaches

The natural greedy algorithm fails to give a constant approximation ratio:

Greedy 1: Pick highest-degree vertex repeatedly

Algorithm 3: Greedy-Degree

```

1: while  $E \neq \emptyset$  do
2:    $v \leftarrow \arg \max_{u \in V} \deg(u)$ 
3:   Add  $v$  to  $C$ ; remove  $v$  and all incident edges
4: end
5: return  $C$ 

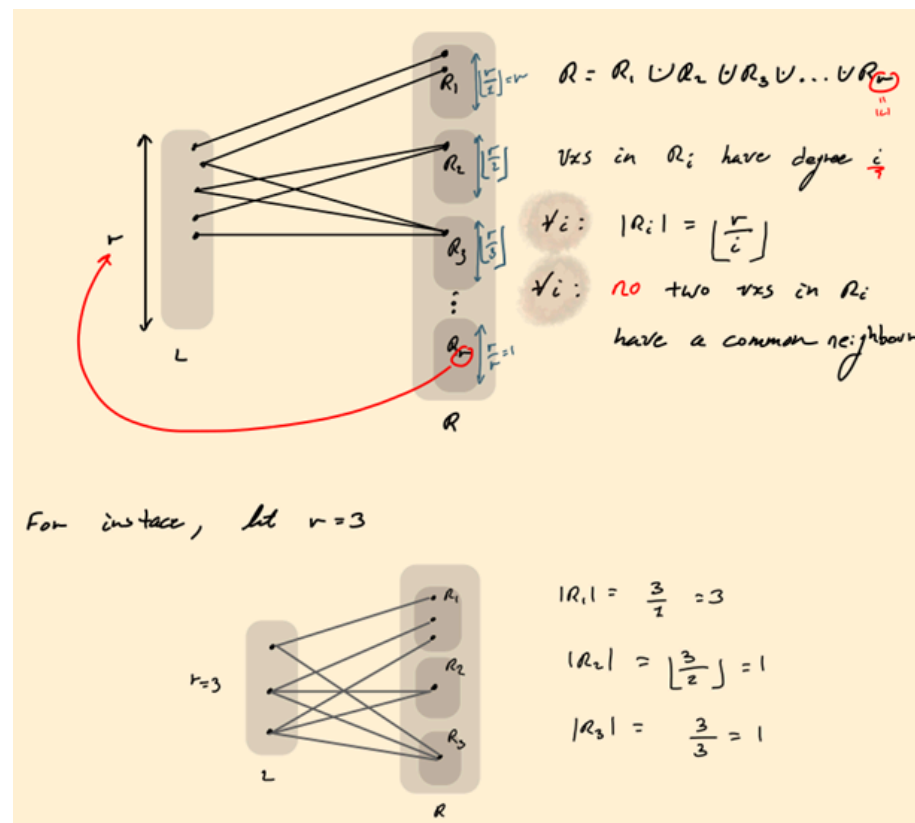
```

Minimum **vc** is the set L of size r .

- The algorithm may pick repeatedly vertices in R_1 then $R_2, R_3 \dots$ returning the set R .
- $|R| = \sum_{i=1}^r |R_i| = \sum_{i=1}^r \left\lfloor \frac{r}{i} \right\rfloor \approx r \cdot \sum_{i=1}^r \frac{1}{i} = \Theta(r \log r)$.
- Approximation ratio of $|R|/|L| = \Omega(\log r)$

The Greedy algorithm is $\Theta(\log r)$ approximation

We can do better!



2. 2-Approximation via Max Matching

Observation 4 Let M be any **maximal matching** in G . Then $V(M)$ (the endpoints of M) is a vertex cover, and $|V(M)| = 2|M|$.

Observation 5 Let M be any **maximal matching** in G . Then $V(M)$ (the endpoints of M) is a vertex cover, and $|V(M)| = 2|M|$.

Algorithm 5: Approx-VC

```
1:  $M \leftarrow$  a maximal matching of  $G$ 
2: return  $V(M)$ 
```

Observation 6 Let M be any **maximal matching** in G . Then $V(M)$ (the endpoints of M) is a vertex cover, and $|V(M)| = 2|M|$.

Algorithm 6: Approx-VC

```

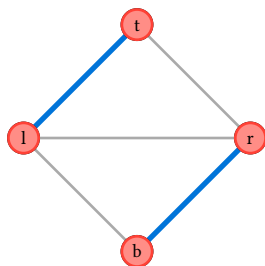
1:  $M \leftarrow$  a maximal matching of  $G$ 
2: return  $V(M)$ 
    
```

Theorem 3 Approx-VC is a 2-approximation for Vertex Cover.

Proof. Let C^* be an optimal vertex cover.

- **Validity:** Every edge in G is either in M or shares an endpoint with some edge in M (by maximality). Either way it is covered by $V(M)$.
- **Ratio:** Every edge $e \in M$ must have at least one endpoint in C^* , and matching edges are **disjoint**, so $|C^*| \geq |M|$. Thus

$$|V(M)| = 2|M| \leq 2|C^*|. \quad \square$$



- Blue edges form a maximal matching M
- $V(M) = \{t, l, r, b\}$ is the returned cover
- Optimal cover: $\{l, r\}$ (size 2)
- $|V(M)| = 4 = 2 \cdot |C^*|$ – tight example

□

3. Savage's Algorithm

Algorithm 7: Savage-VC

```
1:  $T \leftarrow$  a DFS tree of  $G$   
2: return  $I(T) :=$  all internal (non-leaf) vertices of  $T$ 
```

Trivially: Every edge in G has an endpoint in $I(T) \implies I(T)$ is a vertex cover.

Algorithm 8: Savage-VC

```
1:  $T \leftarrow$  a DFS tree of  $G$   
2: return  $I(T) :=$  all internal (non-leaf) vertices of  $T$ 
```

Trivially: Every edge in G has an endpoint in $I(T) \implies I(T)$ is a vertex cover.

Theorem 6 Savage's algorithm is a 2-approximation for Vertex Cover.

It suffices to prove:

Lemma 7 Let T be a DFS tree and $I(T)$ its internal vertices. Then T contains a matching of size $\geq \frac{|I(T)|}{2}$.

Algorithm 9: Savage-VC

- 1: $T \leftarrow$ a DFS tree of G
 - 2: **return** $I(T) :=$ all **internal** (non-leaf) vertices of T
-

Trivially: Every edge in G has an endpoint in $I(T) \implies I(T)$ is a vertex cover.

Theorem 8 Savage's algorithm is a 2-approximation for Vertex Cover.

It suffices to prove:

Lemma 9 Let T be a DFS tree and $I(T)$ its internal vertices. Then T contains a matching of size $\geq \frac{|I(T)|}{2}$.

Lemma $\implies$ Theorem: The matching in T is also a matching in G , so $\tau(G) \geq \nu(G) \geq \frac{|I(T)|}{2}$. The algorithm returns $I(T)$, so $|I(T)| \leq 2\tau(G)$. $\square$

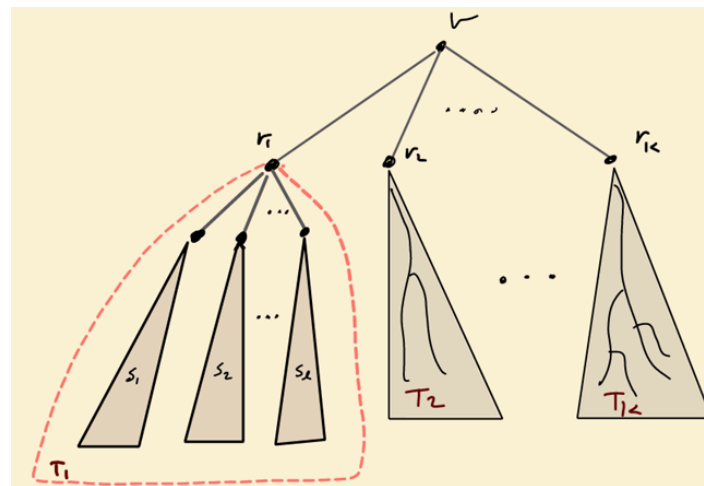
Proof. By induction on $e(T)$. Let r be the root, $r_1, \dots, r_k$ its children, $T_1, \dots, T_k$ the subtrees rooted at them. For T_1 : let $S_1, \dots, S_\ell$ be the subtrees rooted at the **children** of r_1 .

By the IH, T contains a matching of size:

$$\geq \underbrace{1}_{r-r_1} + \sum_{j=1}^{\ell} \frac{|I(S_j)|}{2} + \sum_{i=2}^k \frac{|I(T_i)|}{2}$$

Since $\sum_j |I(S_j)| + \sum_{i \geq 2} |I(T_i)| = |I(T)| - 2$ (r and r_1 are internal but appear in neither sum), the matching has size $\geq 1 + \frac{|I(T)| - 2}{2} = \frac{|I(T)|}{2}$. $\square$

$\square$



4. Minimum Weight Vertex Cover

Weighted VC: Each vertex v has weight $w_v \geq 0$. Minimize $\sum_v w_v x_v$ subject to covering all edges.

Weighted VC: Each vertex v has weight $w_v \geq 0$. Minimize $\sum_v w_v x_v$ subject to covering all edges.

Integer Program:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad x_v \in \{0, 1\}.$$

Weighted VC: Each vertex v has weight $w_v \geq 0$. Minimize $\sum_v w_v x_v$ subject to covering all edges.

Integer Program:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad x_v \in \{0, 1\}.$$

LP Relaxation:

Weighted VC: Each vertex v has weight $w_v \geq 0$. Minimize $\sum_v w_v x_v$ subject to covering all edges.

Integer Program:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad x_v \in \{0, 1\}.$$

LP Relaxation: Replace $x_v \in \{0, 1\}$ with $x_v \in [0, 1]$:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad 0 \leq x_v \leq 1.$$

Weighted VC: Each vertex v has weight $w_v \geq 0$. Minimize $\sum_v w_v x_v$ subject to covering all edges.

Integer Program:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad x_v \in \{0, 1\}.$$

LP Relaxation: Replace $x_v \in \{0, 1\}$ with $x_v \in [0, 1]$:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad 0 \leq x_v \leq 1.$$

Algorithm 14: LP-Round-VC

- 1: $x^* \leftarrow$ optimal LP solution
 - 2: **return** $C = \{v \in V : x_v^* \geq \frac{1}{2}\}$
-

Weighted VC: Each vertex v has weight $w_v \geq 0$. Minimize $\sum_v w_v x_v$ subject to covering all edges.

Integer Program:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad x_v \in \{0, 1\}.$$

LP Relaxation: Replace $x_v \in \{0, 1\}$ with $x_v \in [0, 1]$:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad 0 \leq x_v \leq 1.$$

Algorithm 15: LP-Round-VC

- 1: $x^* \leftarrow$ optimal LP solution
 - 2: **return** $C = \{v \in V : x_v^* \geq \frac{1}{2}\}$
-

Lemma 20 $C = \{v : x_v^* \geq \frac{1}{2}\}$ is a valid vertex cover.

Proof. For any edge $\{u, v\} \in E$: $x_u^* + x_v^* \geq 1$, so $\max(x_u^*, x_v^*) \geq \frac{1}{2}$. Hence at least one of u, v is in C . $\square$

$\square$

Weighted VC: Each vertex v has weight $w_v \geq 0$. Minimize $\sum_v w_v x_v$ subject to covering all edges.

Integer Program:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad x_v \in \{0, 1\}.$$

LP Relaxation: Replace $x_v \in \{0, 1\}$ with $x_v \in [0, 1]$:

$$\min \sum_{v \in V} w_v x_v, \quad x_u + x_v \geq 1; \quad \forall \{u, v\} \in E, \quad 0 \leq x_v \leq 1.$$

Algorithm 16: LP-Round-VC

1: $x^* \leftarrow$ optimal LP solution
2: **return** $C = \{v \in V : x_v^* \geq \frac{1}{2}\}$

Lemma 22 $C = \{v : x_v^* \geq \frac{1}{2}\}$ is a valid vertex cover.

Proof. For any edge $\{u, v\} \in E$: $x_u^* + x_v^* \geq 1$, so $\max(x_u^*, x_v^*) \geq \frac{1}{2}$. Hence at least one of u, v is in C . □

Theorem 23 LP-Round-VC is a 2-approximation for weighted vertex cover.

Proof.

$$w(C) = \sum_{v: x_v^* \geq \frac{1}{2}} w_v \leq \sum_v w_v \cdot (2x_v^*) = 2 \cdot \underbrace{\sum_v w_v x_v^*}_{\text{cost of optimal LP solution}} \leq 2 \cdot \text{OPT}. \quad \square$$

□

5. A Randomized Algorithm

Goal: A randomized 2-approximation for (unweighted) Vertex Cover.

Goal: A randomized 2-approximation for (unweighted) Vertex Cover.

Algorithm 18: Rand-VC

```
1:  $C \leftarrow \emptyset$ 
2: while  $E \neq \emptyset$  do
3:   Pick any edge  $\{u, v\} \in E$ 
4:   Uniformly at random pick  $x \in \{u, v\}$  and set  $C = C \cup \{x\}$ .
5:   Set  $E = E \setminus \{e \in E : x \in e\}$ .
6: end
7: return  $C$ 
```

Goal: A randomized 2-approximation for (unweighted) Vertex Cover.

Algorithm 19: Rand-VC

```
1:  $C \leftarrow \emptyset$ 
2: while  $E \neq \emptyset$  do
3:   Pick any edge  $\{u, v\} \in E$ 
4:   Uniformly at random pick  $x \in \{u, v\}$  and set  $C = C \cup \{x\}$ .
5:   Set  $E = E \setminus \{e \in E : x \in e\}$ .
6: end
7: return  $C$ 
```

Theorem 26 Rand-VC is a randomized 2-approximation: $\mathbb{E}[|C|] \leq 2|C^*|$.

- $C^* :=$ an optimal VC
- $C :=$ the cover produced by the algo
- $|C|$ is a random variable (The size of the random VC is random)

Goal: prove that $\mathbb{E}[|C|] \leq 2|C^*|$.

Each round during the edge $uv \in E(G)$ the algo flips a fair coin:

- Define a coin toss **good** if $x \in C^*$
 - $\Pr(x \in C^*) \geq \frac{1}{2}$
- # of good tosses is at most $|C^*|$

Define X the random variable that counts the number of coin tosses required to get to $|C^*|$ **good** coin tosses.

- $E[X] \geq E[|C|]$

Define $X_i :=$ # of tosses needed to get the i 'th **good** toss.

- $X = X_1 + X_2 + \dots + X_{|C^*|}$

By linearity of expectation

$$\mathbb{E}(X) = \sum_{i=1}^{|C^*|} \mathbb{E}(X_i)$$

- $\forall i : X_i \sim \text{Geo}(1/2) \Rightarrow \mathbb{E}(X_i) = 2$.

$\Rightarrow$

$$\mathbb{E}(|C|) \leq \mathbb{E}(X) = \sum_{i=1}^{|C^*|} \mathbb{E}(X_i) \leq 2|C^*|.$$

(if both $u, v \in C$ then with probability 1 we are “good”).
(Otherwise $C^* \subseteq C$ so that C is a VC.)

(in X each success has probability exactly $\frac{1}{2}$ in C its $\geq \frac{1}{2}$.)

□